

T0.2 — Diagnostico corregido

Revision post-hallazgo · Fecha: 2026-05-01 · Modo: solo lectura. · Sin cambios, sin commits, sin push.

Hallazgo nuevo del owner: en Stripe Dashboard solo hay un webhook activo apuntando a <https://www.usadona.com/api/stripe/webhook>. No hay webhook configurado al backend Render. Esto cambia varios supuestos del diagnostico anterior.

1. Backend /webhook/stripe esta siendo usado hoy?

No. Tres evidencias:

- Stripe Dashboard apunta a [https://www.usadona.com/...](https://www.usadona.com/), no a Render. El backend en Render no recibe webhooks de Stripe hoy.
- Stripe NO permite enviar el mismo evento a dos endpoints simultaneos desde un solo webhook configurado. Si solo hay un webhook listado y va al landing, el backend esta sin trafico.
- El endpoint backend esta vivo en codigo (agent/main.py:2112-2151) y es **publico**. No requiere autenticacion HTTP. El fallback de firma sigue activo.

Conclusion: /webhook/stripe en backend es **dead code en trafico real**, pero **superficie de ataque publica** abierta.

2. El flujo actual depende del webhook de la landing?

Si — y el flujo actual es **muy delgado**:

landing/app/api/webhook/route.ts — recibe checkout.session.completed:

- Loguea 'Payment successful for <customer_email> — plan: <plan>'.
• Si hay metadata.phone o customer_details.phone, manda un unico mensaje WhatsApp via Whapi: 'Hola! Soy Dona, tu nueva asistente. Tu suscripcion esta activa...'.
• **NO acredita creditos**. No llama al backend Python. No persiste la suscripcion.

landing/app/api/webhook/route.ts:20 ya tiene STRIPE_WEBHOOK_SECRET! (non-null assertion: throw si no esta) y rechaza con 400 si falta firma o si constructEvent falla. **A nivel codigo del landing, T0.2 ya esta implementado funcionalmente.**

3. Hay conexion landing -> backend para acreditar creditos?

No. Verifique tres caminos posibles:

- landing/lib/whatsapp.ts — solo Whapi directo, no llama al backend.
- landing/app/api/webhook/route.ts — no hace fetch al backend.
- landing/app/api/checkout/route.ts — crea Checkout Session en Stripe; metadata {plan, phone}. No notifica al backend.
- No hay landing/middleware.ts ni vercel.json con rewrites a dona-agent.onrender.com.

Realidad operativa actual

Cliente paga \$20/\$40 mensual en Stripe	Resultado
Stripe cobra al cliente	OK
Stripe envia checkout.session.completed a webhook configurado	depende de la URL real
Welcome WhatsApp (si webhook llega)	OK (via Whapi en landing)
Creditos acreditados en backend	ninguno
Suscripcion persistida en DB del backend	ninguno
Plan/tier sincronizado en backend	ninguno

El backend Python no se entera de los pagos. El cliente paga la suscripcion pero el sistema de creditos sigue en cero hasta que alguien hace /admin/seed-creditos manual.

4. Que endpoint deberia recibir los eventos en la arquitectura actual?

Hoy en codigo existen dos handlers de Stripe:

- landing/app/api/webhook/route.ts → ruta publica https://www.usadona.com/api/webhook. Funcional pero solo manda welcome.
- agent/main.py:/webhook/stripe → ruta publica https://<render>/webhook/stripe. Funcional con fallback inseguro; sin trafico.

Lo que dijiste:

Stripe Dashboard apunta a https://www.usadona.com/api/stripe/webhook.

Realidad del repo:

NO existe landing/app/api/stripe/webhook/route.ts. El unico webhook handler en la landing esta en landing/app/api/webhook/.

Tres escenarios posibles (necesitan tu confirmacion):

Escenario	Que pasa hoy	Probabilidad
A. La URL en Stripe Dashboard es /api/stripe/webhook literal	Stripe entrega a una ruta que devuelve 404. Ningun evento se procesa. Welcome WhatsApp NO se envia.	Alta si copiaste la URL textual del Dashboard.
B. La URL en Stripe Dashboard es /api/webhook y mencionaste /api/stripe/webhook por confusion visual	El landing recibe el evento y manda el welcome.	Media.
C. Hay un rewrite Vercel server-side (no en este repo) que mapea /api/stripe/webhook -> /api/webhook	Funciona como B.	Baja. No hay next.config.ts/middleware.ts/vercel.json con rewrites en el repo.

Recomendacion inmediata: verificar tres cosas en Stripe Dashboard antes de cualquier cambio de codigo:

- La URL exacta del endpoint (copy-paste, no a vista).
- La pestana 'Events' / 'Webhook attempts' del endpoint — ahi ves si los ultimos eventos respondieron 200 (OK) o 404 (URL muerta).
- Si los ultimos welcome WhatsApp post-pago efectivamente llegaron a los compradores. Si no, escenario A confirmado.

5. T0.2 al backend, al landing, o a ambos?

Componente	Estado de T0.2	Accion
Landing /api/webhook	Ya implementado funcionalmente. STRIPE_WEBHOOK_SECRET! con non-null assertion + constructEvent que rechaza firma invalida.	Nada de codigo. Solo verificar que la env var este en Vercel (no en Render).

Componente	Estado de T0.2	Accion
Backend /webhook/stripe	No implementado. Fallback inseguro presente. Endpoint publico sin trafico.	T0.2 sigue aplicando. La justificacion cambia: no es para 'blindar webhook activo' sino para cerrar superficie de ataque publica.

T0.2 sigue valiendo la pena en el backend porque:

- El endpoint /webhook/stripe esta publicamente expuesto en Render.
- agent/billing.py:procesar_evento_stripe esta conectado al sistema real de creditos (acreditar() y cobrar() siguen vivos).
- Un atacante que descubra la URL puede acreditar creditos forjados a su telefono y luego usar tools creativas (imagen/video/voz) que si funcionan en produccion.

Riesgo P0 sigue abierto aunque el webhook de Stripe no este apuntando ahi.

6. Riesgo real actual si solo existe webhook en usadona.com/api/stripe/webhook

Riesgos operativos (no de seguridad)

- **Si la URL es 404 (escenario A):** el negocio cobra suscripciones pero nadie procesa el checkout.session.completed → **ningun cliente recibe el welcome WhatsApp**, y como el backend tampoco acredita creditos, el cliente paga y queda sin nada visible. **Riesgo de churn / chargebacks / soporte.**
- **Si la URL es correcta (B/C):** welcome funciona. Pero los creditos siguen sin acreditarse en backend (no hay nadie que llame a acreditar).

Riesgos de seguridad (independientes de la URL del Dashboard)

- **/webhook/stripe en Render con fallback inseguro:** P0 abierto. Cualquiera que descubra `https://<render>/webhook/stripe` puede mandar payload forjado y le acreditan creditos.
- **/api/webhook en Vercel:** P0 cerrado funcionalmente (firma obligatoria por non-null assertion + constructEvent).

Riesgo combinado

- **Ya existe un cliente que pago.** Si ningun sistema acredita creditos, la 'compra' en terminos de producto no se materializa. Independiente de T0.2, hay un **gap funcional** entre 'Stripe cobro' y 'Dona entrega'.

7. Plan corregido minimo y seguro

Lo divido en cuatro bloques con prioridades distintas. **No mezclar** en un solo PR.

Bloque 1 — Verificacion operativa (sin codigo, prioridad alta)

Antes de cualquier PR:

- [] Confirmar URL exacta del webhook en Stripe Dashboard (copy-paste). Es `/api/stripe/webhook` o `/api/webhook`?
- [] Revisar en Stripe Dashboard -> Webhook -> 'Events' los ultimos 5 intentos: respondieron 200 o 404?
- [] Verificar con un cliente real reciente si el welcome WhatsApp llego.
- [] Confirmar `STRIPE_WEBHOOK_SECRET` esta seteada en **Vercel** (no en Render) y matchea con el 'Signing secret' del endpoint configurado.

Sin estas respuestas, no se puede decidir si el escenario es A, B o C. El resto del trabajo depende.

Bloque 2 — T0.2 backend (PR de seguridad, prioridad alta)

Sigue siendo valido y necesario. Cierra el riesgo P0 del endpoint publico backend.

Cambios minimos (los del diagnostico anterior):

- `agent/billing.py`: top-level `RuntimeError` si `ENVIRONMENT=production` y no hay `STRIPE_WEBHOOK_SECRET`. Modifica `verificar_firma_stripe` para retornar `None` en produccion sin secret.
- `agent/main.py`: import top-level de `agent.billing` para forzar el check al startup.
- `.env.example`: documentar la variable.
- `tests/test_billing.py`: actualizar/extender.

Cambio respecto al diagnostico anterior:

- La env var `STRIPE_WEBHOOK_SECRET` debe estar en **Render** (backend), pero el valor puede ser **el mismo que esta en Vercel** (mismo Signing secret de Stripe). No hace falta crear un segundo webhook en Stripe — el backend simplemente queda blindado para cuando se decida T1.3 (mover el webhook al backend).
- Si quieres que sea un placeholder distinto y no compartir el secret real entre Vercel y Render, puedes setear en Render un valor distinto (cualquier string `whsec_*` placeholder). El endpoint queda blindado (rechaza todo en produccion) hasta que T1.3 cree un webhook real apuntando a Render.
- **Recomendacion:** compartir el mismo secret. Es mas simple y el dia que migremos el webhook a Render, ya esta listo.

Bloque 3 — Routing landing (operativo, prioridad alta-media)

Independiente de T0.2. Accion depende del escenario:

- **Si A (404):** elegir entre:
 - 3a. Cambiar la URL en Stripe Dashboard a `/api/webhook`. **Cero codigo.**
 - 3b. Crear `landing/app/api/stripe/webhook/route.ts` (identico al actual `/api/webhook` o reutilizando el handler). **PR de landing pequeno.**
- **Si B/C:** ninguna accion.

Bloque 4 — Decision arquitectonica (T1.3 del Plan v2.1)

Despues de B2 + B3, la decision que sigue es:

- **Quien acredita creditos al pagar la suscripcion?** Hoy nadie. Hay que conectar el webhook con `agent/billing.py:acreditar()`.
- **Backend Python como fuente unica (Plan v2.1 §A.2.2)?** Mover el webhook de Stripe del landing al backend, deprecar el welcome del landing, y procesar todo en backend (acreditar + welcome).

Eso ya **no es T0.2 ni Phase 0**. Es Phase 1 (T1.3 + T1.4). Sale fuera del alcance de este reporte.

8. Diferencia con el diagnostico anterior

Aspecto	Diagnostico anterior	Diagnostico corregido
Webhook backend /webhook/stripe	Asumido activo	Confirmado dead code en trafico , pero superficie de ataque publica
Webhook landing /api/webhook	Mencionado como 'independiente'	T0.2 ya implementado funcionalmente ; sin cambios de codigo
URL en Stripe Dashboard	No verificada	Reportada como /api/stripe/webhook — no matchea con codigo
Conexion landing -> backend para acreditar	Implicito	Confirmado: no existe
Justificacion de T0.2 backend	'Blindar webhook activo'	'Cerrar superficie de ataque del endpoint publico sin trafico'
Variable a configurar	STRIPE_WEBHOOK_SECRET en Render	Igual . Recomendacion nueva: mismo valor que Vercel, o placeholder.
Bloques del trabajo	1 (T0.2 backend)	3 (verificacion + T0.2 backend + routing landing) + Bloque 4 fuera de scope

9. Recomendacion de orden

- **Bloque 1 ahora:** investigacion operativa en Stripe Dashboard (5 min, sin codigo). Sin esto, no podes saber si los pagos estan entregando welcome WhatsApp.
- **Bloque 2** despues: T0.2 backend (PR pequeno, riesgo bajo). Cierra P0 sin necesitar Stripe operativo.
- **Bloque 3** en paralelo o despues: si A se confirma, decidir 3a vs 3b. Es 5 min de Dashboard o un PR de landing pequeno.
- **Bloque 4** despues de Phase 0: decision de arquitectura T1.3 (acreditacion de creditos por suscripcion).

10. Resumen ejecutivo

- **Backend /webhook/stripe no recibe trafico hoy** (Stripe no apunta ahi), pero esta publicamente expuesto con fallback inseguro → **P0 vigente**.
- **Landing /api/webhook ya tiene firma obligatoria por codigo**. T0.2 a nivel codigo del landing no requiere PR.
- **Stripe Dashboard apunta a usadona.com/api/stripe/webhook** — esa ruta **no existe en el repo**. Hay alta probabilidad de que los webhooks esten fallando con 404 silenciosamente.

- **El sistema de creditos del backend no se acredita al cobrar suscripciones:** gap funcional independiente de T0.2.
- **T0.2 backend sigue siendo PR valido y necesario**, con justificacion corregida: cerrar superficie de ataque publica.

Estado: no implementado todavia. Esperando confirmacion del owner de:

- (1) URL exacta del webhook en Stripe Dashboard.
- (2) Decision sobre Bloque 3 (arreglar routing) y si avanzo con Bloque 2 (PR backend) en paralelo.